Les instructions conditionnelles

Syntaxe

```
if condition1:
        instruction 1
elif condition2:
        instruction 2
elif condition3:
        instruction 3
        instruction 4
else :
        instruction 5
        instruction 6
```

Bloc d'instructions à exécuter si la condition 1 est vraie.

Bloc d'instructions à exécuter si la condition 2 est vraie.

Bloc d'instructions à exécuter si la condition 3 est vraie.

Bloc d'instructions à exécuter si aucune des conditions précédentes n'est vraie.

PS: Attention à l'indentation

Exemple

```
x=1
#Condition1:
if x>0 :
  print(x,"est positif")
#Condition 2:
elif x<0 :
  print(x,"est négatif")
#Condition 3:
else:
  print(x,"est nul")
```

Les instructions itératives – Boucle for

Syntaxe

```
for i in range(a):
        instructions
for i in range(a,b):
        instructions
for i in range(a,b,c):
        instructions
for i in (liste/chaine de caractere/tuple/dictionnaire/fichier/…..):
        instructions
```

range()=>retourne une séquence de nombres que l'on peut parcourir

Exemple:

range(10)->0,1,2,3,4,5,6,7,8,9

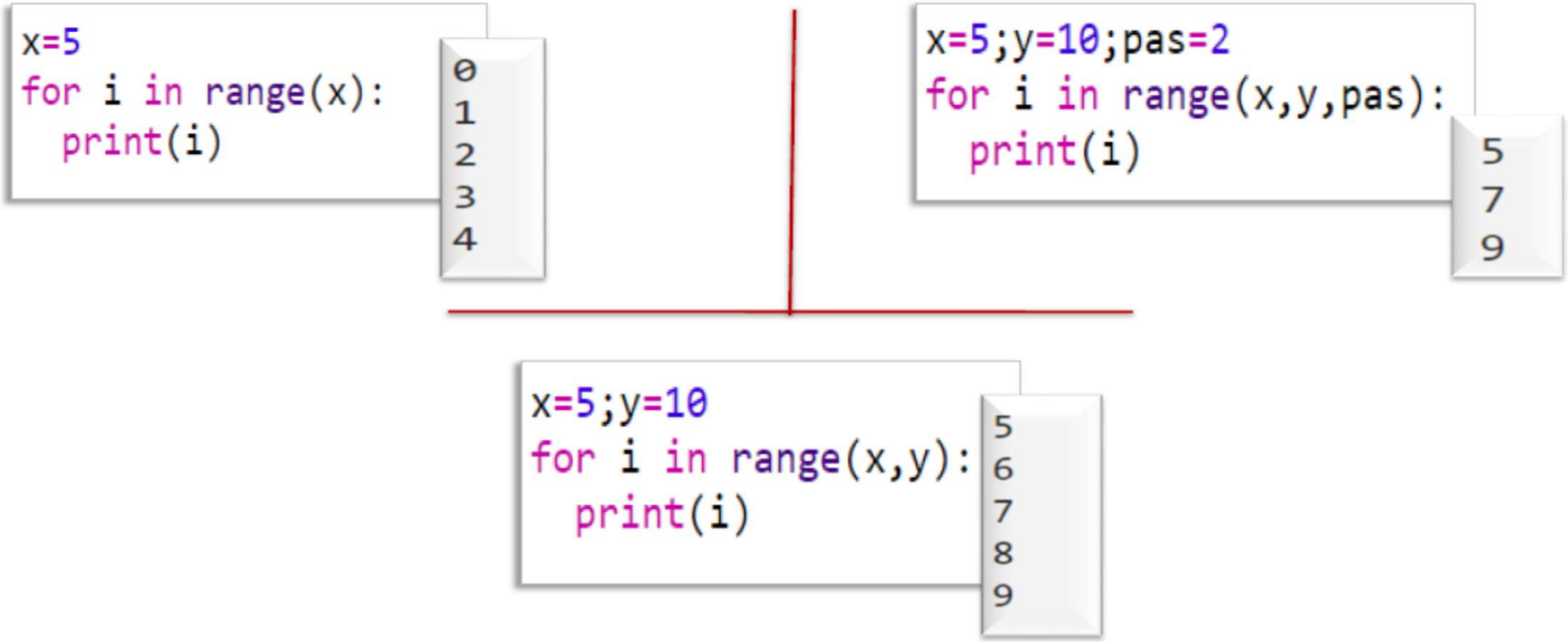
range (a) désigne l'intervalle [0,a[

range (a,b) désigne l'intervalle [a,b[

range (a,b,c) désigne l'intervalle [a,b[par pas entier égal à c

PS: Attention à l'indentation

Les instructions itératives – Boucle for



Les instructions itératives – While

Syntaxe

```
1 | while condition/expression:
2 |     instructions
```

PS: Attention à l'indentation

Exemple

```
1 | nb = 1
2 | while nb <= 5:
3 |     print(nb)
4 |     nb += 1
```

```
1
2
3
4
5
```

Les instructions break, continue et la clause else dans les boucles

L'instruction **break** permet de « casser » l'exécution d'une boucle (while ou for). Elle fait sortir de la boucle et passer à l'instruction suivante

```
34      for i in range(10):
35          print("debut iteration", i)
36          print("bonjour")
37          if i == 2:
38              break
39          print("fin iteration", i)
40      print("apres la boucle")
```

```
In [20]: runfile('C:/Users/Sawssen/python codes/
Users/Sawssen/python codes')
debut iteration 0
bonjour
fin iteration 0
debut iteration 1
bonjour
fin iteration 1
debut iteration 2
bonjour
apres la boucle
```

Note : dans le cas de boucles imbriquées, l'instruction **break** ne fait sortir que de la boucle la plus interne

Les instructions break, continue et la clause else dans les boucles

Remarque : équivalent du 'do … while' (faire…tant que):

Dans de nombreux langages, il existe une instruction **do…while** qui permet de créer une boucle pour laquelle on ne connaît pas à l'avance le nombre d'itérations, mais qui doit s'exécuter au moins une fois. Cette instruction n'existe pas en python, Néanmoins, on peut reproduire son fonctionnement de la manière suivante :

```
while True:
    n = int(input("donnez un entier > 0 : "))
    print("vous avez fourni", n)
    if n > 0:
        break
print("reponse correcte")
```

```
In [18]: runfile('C:/Users/Sawssen/python codes/
Users/Sawssen/python codes')

donnez un entier > 0 : -8
vous avez fourni -8

donnez un entier > 0 : 0
vous avez fourni 0

donnez un entier > 0 : 5
vous avez fourni 5
reponse correcte
```

Les instructions break, continue et la clause else dans les boucles

L'instruction **continue** permet de passer précocement au tour de boucle suivant. Elle fait continuer sur la prochaine itération de la boucle.

```
18    for i in range(4):
19        print('début itération', i)
20        print('bonjour')
21        if i<2:
22            continue
23        print('fin itération',i)
24    print('fin de l exécution de la boucle')
```

```
In [11]: runfile('C:/Users/Sawssen/python codes/s2.py
Users/Sawssen/python codes')
début itération 0
bonjour
début itération 1
bonjour
début itération 2
bonjour
fin itération 2
début itération 3
bonjour
fin itération 3
fin de l exécution de la boucle
```

Les instructions break, continue et la clause else dans les boucles

La clause **else** dans une boucle:

La clause **else** dans une boucle permet de définir un bloc d'instructions qui **sera exécuté à la fin seulement si la boucle s'est déroulée complètement** sans être interrompue par un break (contrairement aux instructions présentes après la boucle).

```
42  for n in range(2, 8):
43      for x in range(2, n):
44          if n % x == 0:
45              print(n, "egale", x, "*", n/x)
46              break
47      else:
48          print(n, "est un nombre premier")
```

```
In [22]: runfile('C:/Users/Sawssen/python codes/s
Users/Sawssen/python codes')
2 est un nombre premier
3 est un nombre premier
4 egale 2 * 2.0
5 est un nombre premier
6 egale 2 * 3.0
7 est un nombre premier
```

Les fonctions: Définition & Appel

Définition d'une fonction

Objectif : structurer un programme en des sous programmes.

Avantages :

- Meilleure organisation du programme (regrouper les tâches par blocs : lisibilité et facilité de maintenance).
- Eviter la redondance et assurer une meilleure réutilisation du code.
- Possibilité de partager les fonctions (via des modules)

Définition d'une fonction

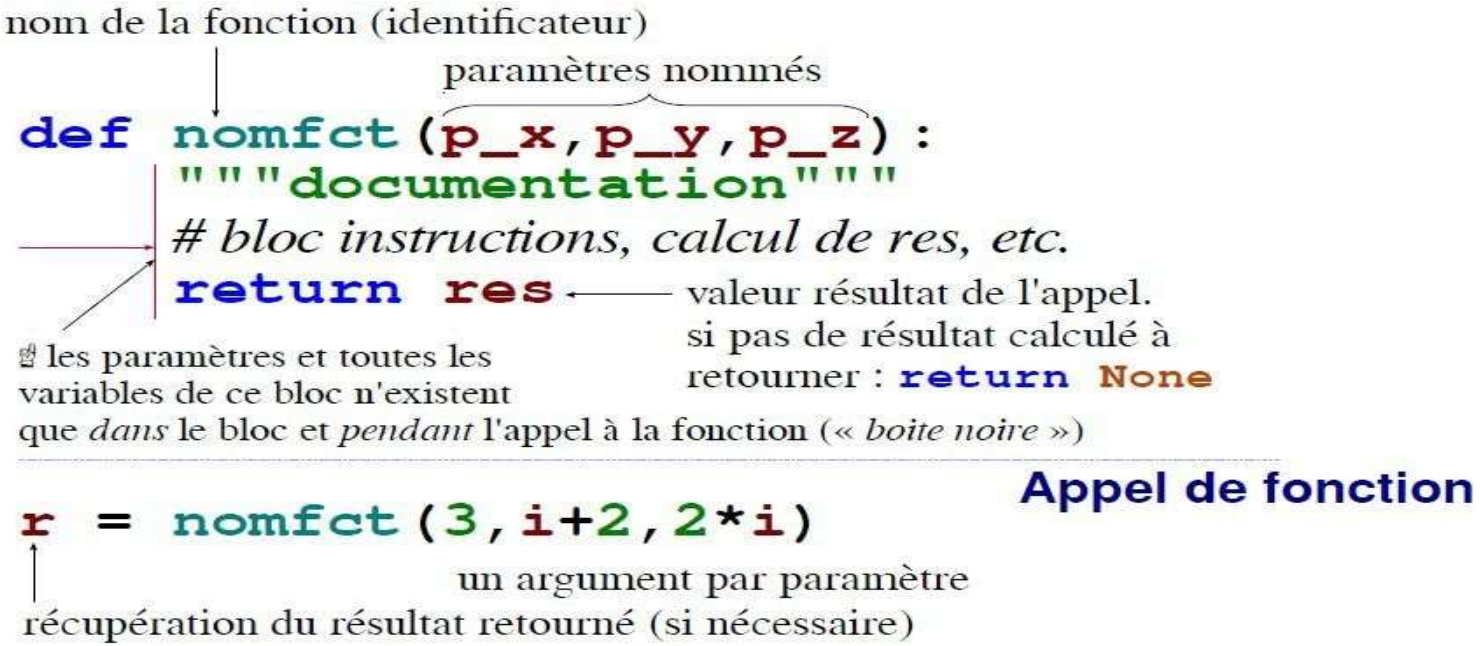
Définition : Une fonction est:

- Un groupe d'instructions regroupées sous un nom et s'exécutant à l'appel.
- Prend des paramètres en entrée (non typés)
- Renvoie une valeur en sortie (ou plusieurs valeurs, ou pas de valeurs du tout : procédure)

Syntaxe :

```
def nom_fonction(liste de paramètres):
    bloc d'instructions
```

Définition d'une fonction



Exemple

```
""" on va faire un programme qui  calcule la division de 2 entiers  """
def division(a,b):
    if(a!=0):
        return a/b
    else:
        return None

a=int(input("Entrez un entier a = "))
b=int(input("Entrez un entier b = "))
d=division(a,b)
if(d!=None):
    print("Résultat de division : ", d)
else:
    print("Erreur ! Division par zéro")
```

Des observations ?

- **Indentation** remplace les {} du C/Java/...
- Typage dynamique

```
In [32]: runfile('C:/Users/Sawssen/python codes/s2.p
Users/Sawssen/python codes')

Entrez un entier a = 5

Entrez un entier b = 2
Résultat de division :  2.5
```

Appel de fonction

mode de passage de paramètres

Les règles de passage des paramètres sont les suivantes :

- Les paramètres **immutables (non modifiables)** correspondent au **passage par valeur**. De fait le passage se fait bien par référence, mais puisqu'il est impossible de changer des objets non modifiable en place, cela revient à faire une copie des données.

- Les paramètres **mutables**, tels les listes, les dictionnaires… correspondent au **passage par référence** (par adresse: **VAR**, **S** et **ES** en algorithmique).

- Dans n'importe quelle fonction on peut inverser les arguments lors d'un appel si on les nomme :

```
1  def puissance(a, b):
2      return a**b
3
4  puissance(2,3)==puissance(b=3,a=2)
```

Portée de variables: locales Vs globales

- Les variables définies localement dans les fonctions sont uniquement visibles dans ces fonctions.

- Les variables définies en dehors de la fonction (dans la mémoire globale) sont accessibles dans la fonction. Mais ne peuvent pas changer de valeur (sauf si on les précède du mot clé global).

```
x=5
def addx(y):
    x=x+y
    return(x)

print(addx(10))
```

```
File "C:\Users\Sawssen\python codes\s2.py", line 72, in <module>
  print(addx(10))

File "C:\Users\Sawssen\python codes\s2.py", line 69, in addx
  x=x+y

UnboundLocalError: local variable 'x' referenced before assignment
```

```
x=5
def addx(y):
    global x
    x=x+y
    return(x)

print(addx(10))
```

```
In [55]: runfile('C:/Users/Sawssen/python codes/
Users/Sawssen/python codes')
15
```